WEEK -4

Task-1:

Process Synchronization and Child Process Monitoring Using waitpid()

kiran@k: ~

GNU nano 8.7.1 task9.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child: Process started\n");

        sleep(2);

        printf("Child: Process completed\n");
        exit(5);
    }
    else {
        printf("Parent: Waiting for child...\n");

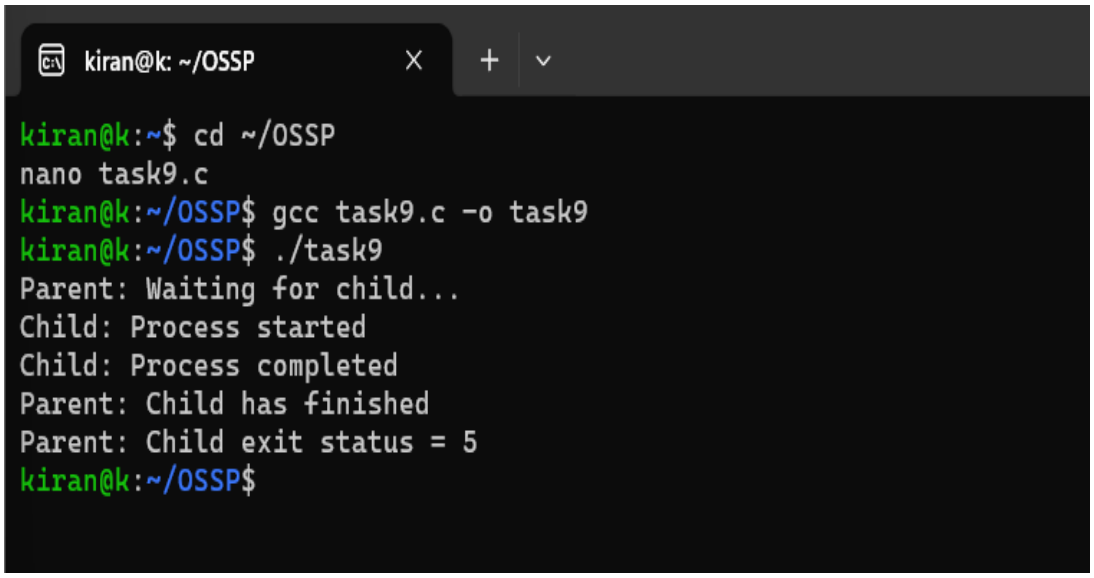
        waitpid(pid, &status, 0);

        printf("Parent: Child has finished\n");

        if (WIFEXITED(status)) {
            printf("Parent: Child exit status = %d\n",
                   WEXITSTATUS(status));
        }
    }

    return 0;
}
```

OUTPUT:



I created a parent and child process using fork() and used waitpid() to synchronize the child process with the parent. From the output, I observed that the parent waited until the child process completed and then checked its exit status. I understood how waitpid() helps the parent monitor and synchronize with a particular child process.

Task-2:

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

kiran@k: ~/OSSP

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

#define MAX_PATH 4096

int main()
{
    char *path;
    char *path_copy;
    char *directory;
    char command[100];
    char full_path[MAX_PATH];

    printf("Enter command: ");
    scanf("%99s", command);

    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable not found.\n");
        return 1;
    }

    path_copy = strdup(path);

    if (path_copy == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    directory = strtok(path_copy, ":");

    while (directory != NULL)
    {
        snprintf(full_path, MAX_PATH, "%s/%s", directory, command);

        if (access(full_path, X_OK) == 0)
        {
            printf("Command found: %s\n", full_path);
            free(path_copy);
            return 0;
        }

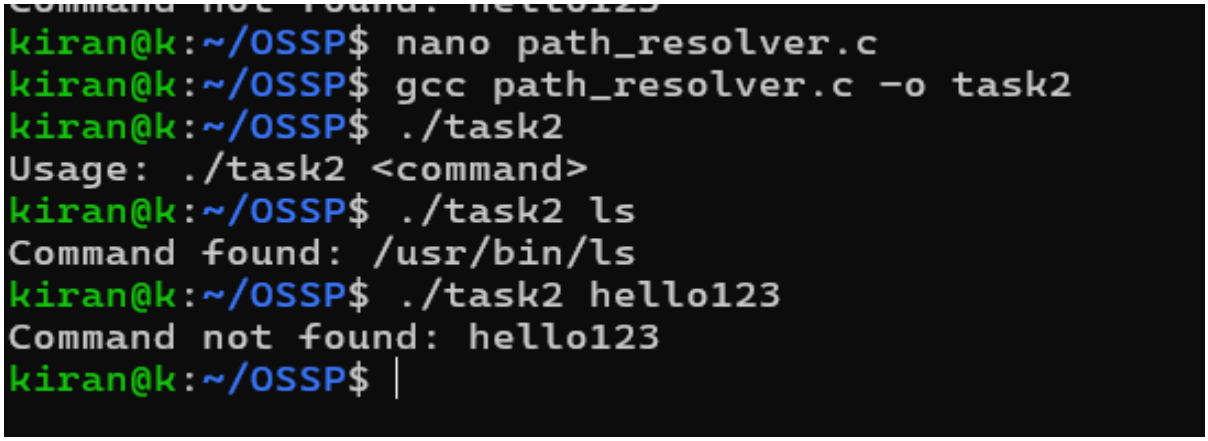
        directory = strtok(NULL, ":");
    }

    printf("Command not found: %s\n", command);

    free(path_copy);

    return 0;
}
```

OUTPUT:



Report:

I implemented a program to search for commands using the PATH environment variable. The program searches the directories in PATH and checks whether the command exists and has executable permission. I tested the program using the ls command and it successfully located the executable. I also tested a missing command to check the error handling. I understood how the shell searches for executable commands using the PATH variable.

Task-3: To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

kiran@k: ~/OSSP

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_OUTPUT 500

void expand_variables(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1)
    {
        if (input[i] == '$')
        {
            i++;

            if (input[i] == '(')
            {
                i++;

                char variable[100];
                int k = 0;

                while (input[i] != '\0' && input[i] != ')' && k < 99)
                {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                if (input[i] == ')')
                    i++;

                char *value = getenv(variable);

                if (value != NULL)
                {
                    int m = 0;

                    while (value[m] != '\0' && j < MAX_OUTPUT - 1)
                    {
                        output[j++] = value[m++];
                    }
                }
                else
                {
                    printf("Undefined variable: %s\n", variable);
                }
            }
            else
            {
                char variable[100];
                int k = 0;

                while (input[i] != '\0' &&
                       (isalnum((unsigned char)input[i]) || input[i] == '_') &&
                       k < 99)
                {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                char *value = getenv(variable);

                if (value != NULL)
                {
                    int m = 0;

                    while (value[m] != '\0' && j < MAX_OUTPUT - 1)
                    {
                        output[j++] = value[m++];
                    }
                }
                else
                {
                    printf("Undefined variable: %s\n", variable);
                }
            }
        }
        else
        {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

int main()
{
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("Enter string: ");
    fgets(input, MAX_INPUT, stdin);

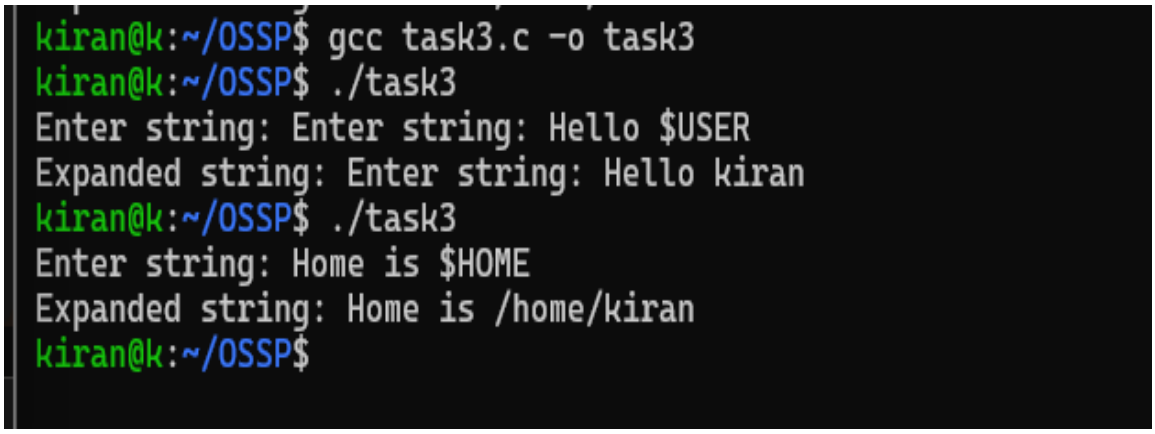
    input[strcspn(input, "\n")] = '\0';

    expand_variables(input, output);

    printf("Expanded string: %s\n", output);

    return 0;
}
```

OUTPUT:



Report:

I implemented variable expansion in the shell by detecting variable references using $ and retrieving their values from the environment. I tested defined and undefined variables and observed that defined variables are replaced with their actual values, while undefined variables are reported. I understood how environment variables can be expanded and updated in the input string.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logical

Task-4:

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

kiran@k: ~/OSSP

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 100

void builtin_cd(char *args)
{
    if (args == NULL)
    {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(args) != 0)
    {
        perror("cd");
    }
}

void builtin_pwd()
{
    char path[1024];

    if (getcwd(path, sizeof(path)) != NULL)
    {
        printf("%s\n", path);
    }
}

void builtin_echo(char *args)
{
    if (args != NULL)
    {
        printf("%s\n", args);
    }
}

void builtin_help()
{
    printf("Built-in commands:\n");
    printf("cd\n");
    printf("pwd\n");
    printf("echo\n");
    printf("help\n");
    printf("exit\n");
}

int main()
{
    char input[MAX_INPUT];

    while (1)
    {
        printf("myshell> ");
        fgets(input, MAX_INPUT, stdin);

        input[strcspn(input, "\n")] = '\0';
```

```c
        char *command = strtok(input, " ");
        char *args = strtok(NULL, "");

        if (command == NULL)
            continue;

        if (strcmp(command, "cd") == 0)
        {
            builtin_cd(args);
        }
        else if (strcmp(command, "pwd") == 0)
        {
            builtin_pwd();
        }
        else if (strcmp(command, "echo") == 0)
        {
            builtin_echo(args);
        }
        else if (strcmp(command, "help") == 0)
        {
            builtin_help();
        }
        else if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }
        else
        {
            printf("Invalid command: %s\n", command);
        }
    }

    return 0;
}
```

OUTPUT:

```
kiran@k:~/OSSP$ gcc task4.c -o task4
kiran@k:~/OSSP$ ./task4
myshell> pwd
/home/kiran/OSSP
myshell> echo Hello
Hello
myshell> cd ..
myshell> pwd
/home/kiran
myshell> abc
Invalid command: abc
myshell> exit
Exiting shell...
kiran@k:~/OSSP$
```

Report:

I implemented a dispatch system for built-in shell commands such as cd, pwd, echo, help, and exit. I tested the commands and also checked an invalid command to verify error handling. I understood that built-in commands are executed within the shell process and that commands like cd can maintain the shell's current state.